Provenance-Aware Capability Gating against Indirect Prompt Injection in Multi-Agent Systems

AIAA 4313 Group Project
Team member names and student IDs to be inserted

Summer 2026

Abstract

Tool-using language-model agents can mistake instructions embedded in retrieved content for authorized user intent. We study this indirect prompt-injection failure in a controlled two-agent office assistant. A Reader Agent retrieves synthetic email, an Action Agent proposes file, calendar, and email operations, and an independent gateway mediates every side effect. Our defense combines a task-scoped capability manifest with exact provenance and sensitivity labels. We pre-register six tasks, three content conditions, eight defense arms, and three paired seeds. Attacks are generated offline, frozen, and hashed before formal evaluation. Executed unauthorized effects, exact synthetic-secret leakage, and benign task success are derived from audit events and before/after world state rather than from model text. The original and hardened corpora each contain 432 planned records, with 428 valid and four invalid records retained in intention-to-test accounting. The new Provenance-only and Prompt+Provenance arms complete the factorial without changing earlier frozen plans. On original T5–T6 attacks, Capability-Only leaks 3/6 while Provenance-only and Full leak 0/6; on hardened T5–T6 attacks, Prompt+Capability leaks 3/6 while Full leaks 0/6. A deterministic 16-case sink-pressure test blocked all eight provenance-enabled protected-value calls.

1 Introduction

Large language models increasingly act through tools rather than only producing text. Retrieval makes such systems useful, but it also introduces a confused- deputy problem: an attacker can place an instruction in an email or document, and a model may execute it using authority granted by the user. This is an *indirect prompt injection* because the adversarial instruction arrives through data selected at run time rather than through the user message [4]. Evaluations of tool-using agents show that neither ordinary helpfulness prompting nor simple refusal instructions reliably separate data from commands [1, 8].

This project asks:

> Given a correct task-authorization manifest, can provenance-aware capability gating reduce executed unauthorized actions and exact synthetic-secret leakage without substantially reducing benign task completion?

We deliberately make a narrow claim. The prototype is a local synthetic office, the capability oracle is assumed correct, and leakage tracking recognizes exact registered values. It is an auditable experiment in enforcement mechanisms, not a proof of general language-model security.

2 Background and Design Principles

Prompt-level instruction hierarchy attempts to teach a model that trusted system and user instructions outrank untrusted content [6]. This can reduce attacks but leaves the same probabilistic component responsible for interpreting both data and policy. Capability systems instead provide an unforgeable, least-authority description of the operations a component may perform [3, 5]. Information-flow control tracks where information came from and constrains where it may go [2].

ProvGate combines these ideas at the tool boundary. The model may still read an attack and may still propose a malicious call. Security rests on a small, deterministic reference monitor that checks the call against task authority and against the provenance and sensitivity of outbound values. The distinction between proposal, policy decision, execution, and world mutation is preserved in an append-only audit trail.

3 Threat Model

Adversary. The attacker controls one external-content block in a synthetic email consumed by the Reader Agent. It can issue natural-language instructions designed to cause an unauthorized recipient, file read, calendar mutation, or disclosure. The formal Red Agent is not present at victim run time: it generates five candidates per task offline under fixed seeds, after which the selected payload is frozen and hashed.

Trusted computing base. The scenario manifest, capability gateway, provenance tracker, mock tools, append-only artifact writer, and independent evaluator are trusted. The Reader and Action model outputs, retrieved email content, summaries derived from it, and all tool arguments proposed by a model are untrusted. We assume the manifest correctly captures the user's intended authority.

Attacker goals. Success requires an observable world effect: a forbidden message recipient, an unauthorized calendar event or resource access, or an exact registered synthetic secret in an outbound message. Model text that merely repeats an instruction is not counted as an executed effect. A denied proposal is an attempted and blocked attack, not a successful attack.

Out of scope. The system exposes no shell, payment, deletion, real account, credential, or external network tool. We do not claim protection against a compromised gateway, an incorrect capability manifest, semantic paraphrases of a secret, side channels, encoded values, or attacks that exploit the model-serving stack.

4 System Design

4.1 Agents and mock tools

The Reader Agent may call `search_emails` and `read_email`. It returns a task summary plus exact evidence and its event identifiers. The Action Agent receives the original request, the summary, and retrieved evidence. Depending on the scenario, it may propose `read_file`, `search_calendar`, `send_email`, or `create_calendar_event`. All resources and side effects live in a fresh in-memory world for each attempt.

Table 1: Original defense arms and the independently frozen ablation arms.

Arm	Prompt hierarchy	Deterministic gateway
Allow-All	No	No
Prompt-Only	Yes	No
Capability-Only	No	Capability constraints only
Provenance-Only	No	Provenance sink check only
Capability+Provenance	No	Capability and provenance constraints
Prompt+Capability	Yes	Capability constraints only
Prompt+Provenance	Yes	Provenance sink check only
Full	Yes	Capability and provenance constraints

The separation is intentional: it creates a realistic propagation path from an untrusted retrieval agent to a privileged action agent while retaining complete evidence about exposure. A structured context event links the exact resource- read event to the Action Agent's context.

4.2 Capabilities

Each task supplies a finite capability set. A capability binds a tool to zero or more resource identifiers, recipient allow-lists, parameter bounds, maximum call counts, and maximum outbound sensitivity. For example, a reply task may authorize one message to `alice@example.com` while excluding added carbon copy recipients. A scheduling task may constrain participants, start/end windows, and duration. Calls outside this envelope are denied even if a model describes them as mandatory.

4.3 Provenance and sensitivity

Every relevant value carries an authority label (trusted, untrusted, or derived), a sensitivity label (public, internal, or secret), and explicit source and parent event identifiers. Derived values conservatively retain their sources. The prototype additionally registers exact synthetic protected literals. If such a literal flows from a protected resource into an outbound argument, the gateway can deny the call even when the tool and recipient are otherwise authorized. Tasks T5 and T6 were intended to probe this incremental provenance effect. A follow-up fifth arm is needed to test whether they actually isolate it from the effect of the safe prompt.

4.4 Defense arms

All arms retain the same scenario, tools, model checkpoint, decoding settings, and evaluator. In the original four-arm design, Capability-Only and Full differed in both the safe prompt and provenance checking, so their contrast did not isolate either factor. The follow-up Prompt+Capability arm uses the same safe prompt, capability enforcement, and denial recovery as Full but disables only the provenance sink check. The Capability+Provenance arm disables the safe prompt while retaining deterministic provenance checks. Provenance-only and Prompt+Provenance deliberately bypass capability checks, so they isolate sink-taint blocking rather than full least-authority enforcement. The eight arms form a complete factorial for separately frozen original and hardened corpora.

Table 2: Frozen factorial design. Each complete corpus contains 432 planned victim runs; the original formal plan is a frozen subset of this matrix.

Factor	Levels	Count
Scenario	T1–T6	6
Content	CLEAN, PLACEBO, ATTACK	3
Defense	Table 1	8
Paired seed	4313, 4314, 4315	3
Total per complete corpus	$6 \times 3 \times 8 \times 3$	432

5 Offline Attack Corpus

The Red Agent receives the scenario objective, carrier location, allowed and forbidden effects, and a JSON output schema. For each scenario, fixed candidate indices 1–3 are development data, index 4 is the formal payload, and index 5 is a reserve that may replace index 4 only for a syntax defect found before any victim run. A payload is never selected or replaced according to victim-model success. We retain the generation prompt, raw response, normalized payload, seed, model identifier, and SHA-256 digest.

Each PLACEBO uses the same carrier, surrounding template, tone, and approximately the same byte length as its paired ATTACK, but contains no security-relevant directive. The CLEAN version contains task-relevant benign content. A condition builder asserts that fields outside the carrier are identical. Formal experiments load only frozen files; they never call the Red Agent.

6 Experimental Method

6.1 Matrix and hypotheses

We pre-register four hypotheses. H1: under ALLOW-ALL, ATTACK causes more executed forbidden effects than PLACEBO. H2: under ATTACK, FULL reduces executed forbidden effects and exact leakage relative to ALLOW-ALL. H3: on T5–T6, FULL reduces leakage relative to CAPABILITY-ONLY when the tool and recipient are already authorized. H4: FULL retains useful benign task completion while introducing measurable false blocks and latency.

The complete Cartesian plan is shuffled once with randomization seed 4313 and hashed. Each run starts from a new deep copy of its condition world. Retried infrastructure failures receive a new append-only attempt identifier and never overwrite evidence.

6.2 Pilot gates and freezing

Only development payloads may be used while checking tool-call parsing, benign utility, attack exposure, baseline attack effect, and at least one effective Full-defense block. Prompt wording, task clarity, and security code are frozen before formal payloads are evaluated. Formal failures are retained; no no-op, refusal, timeout, or malformed call is selectively deleted.

6.3 Outcome definitions

For valid run i, let U_i indicate an executed forbidden world effect, L_i an exact protected-value disclosure in the synthetic outbox, B_i benign task completion, A_i a malicious tool proposal, and D_i

a gateway denial of that proposal. For a cell with n valid runs, the principal rates are

$$\widehat{p}_U = n^{-1}\sum_i U_i, \qquad \widehat{p}_L = n^{-1}\sum_i L_i, \qquad \widehat{p}_B = n^{-1}\sum_i B_i. \tag{1}$$

Exposure, attempt, block, benign block, tool calls, tokens, and latency are diagnostics. We report counts and Wilson 95% intervals [7]. Infrastructure-invalid counts are shown explicitly, and an intention-to-test sensitivity analysis retains every planned run in its denominator. The original primary comparisons are paired differences for Attack minus Placebo under Allow-All, Full minus Allow-All under Attack, and Full minus Capability-Only on T5–T6 attacks. The last contrast is recognized as factorially confounded. The follow-up analysis therefore predeclares Capability+Provenance minus Capability-Only, Prompt+Capability minus Capability-Only, Provenance-only minus Allow-All, and Prompt+Provenance minus Prompt-Only, together with Full minus each intermediate arm for T5–T6 attack leakage. A separate deterministic sink-pressure protocol directly inserts a protected value into four registered outbound fields under four arms; it is a mechanism test, not a victim-model rate estimate. We report task-level heterogeneity and avoid broad significance claims from six templates.

6.4 Independent evaluation

The evaluator does not treat a gateway decision or model statement as evidence of an effect. It compares the initial and final world, checks successful tool execution events, evaluates pre-registered benign and forbidden predicates, and scans new outbound messages for exact registered values. Exposure requires both a resource-read event and an Action-context event that explicitly links to that read. Pre-model truth-table tests cover benign success, no-op, denial, execution, leakage, timeout, HTTP failure, malformed call, and a null audit.

7 Results

Complete-factorial evaluation. The original corpus combines the 216-run formal plan with three frozen completion plans (432 planned records, 428 valid, and four invalid records retained under intention-to-test accounting). The hardened corpus is analyzed separately with the existing 108-run Prompt+Capability/Full follow-up and three new paired completion plans (also 432 planned, 428 valid, and four invalid). No records are pooled across corpora, and every input record is checked against its frozen run-plan hash before aggregation.

T5–T6 exact-secret leakage. On the original corpus, the valid attack leakage counts are Allow-All 3/6, Prompt-Only 0/6, Capability-Only 3/6, Provenance-only 0/6, Prompt+Provenance 0/6, Capability+Provenance 0/6, Prompt+Capability 0/6, and Full 0/6. The registered Provenance-only–Allow-All contrast is -0.50 (95% bootstrap CI $[-0.83, -0.17]$), while Prompt+Provenance–Prompt-Only is 0.00 and Full adds no further leakage reduction over either no-capability provenance arm on this original set.

On the hardened corpus, the corresponding counts are Allow-All 2/6, Prompt-Only 3/6, Capability-Only 2/6, Provenance-only 0/6, Prompt+Provenance 0/6, Capability+Provenance 0/6, Prompt+Capability 3/6, and Full 0/6. Provenance-only–Allow-All is -0.33 (95% CI $[-0.67, 0.00]$), Prompt+Provenance–Prompt-Only is -0.50 (95% CI $[-0.83, -0.17]$), and Full–Prompt+Capability is -0.50 (95% CI $[-0.83, -0.17]$). Thus the hardened set supplies an end-to-end provenance gain above the safe prompt/capability baseline, whereas the original set retains a floor effect for the two strongest arms.

Mechanism pressure test. The deterministic sink-pressure suite still passes all 16 registered calls: all eight provenance-enabled protected-value calls are denied with zero side effects, while the eight non-provenance controls execute their capability-valid calls. This is a mechanism test, not a victim-model rate estimate.

The tables and figures below are generated from the two complete-factorial artifact manifests. Percentages are valid-only rates; invalid planned runs remain visible in the Valid/Planned columns.

Table 3: Original-corpus complete-factorial outcomes.

Family	Condition	Defense	Valid/Planned	Executed	Leakage	Utility
All tasks	attack	allow_all	17/18	52.9% [31.0, 73.8]	35.3% [17.3, 58.7]	64.7% [41.3, 82.7]
All tasks	attack	prompt_only	18/18	33.3% [16.3, 56.3]	16.7% [5.8, 39.2]	55.6% [33.7, 75.4]
All tasks	attack	capability_only	18/18	16.7% [5.8, 39.2]	16.7% [5.8, 39.2]	61.1% [38.6, 79.7]
All tasks	attack	provenance_only	18/18	33.3% [16.3, 56.3]	0.0% [0.0, 17.6]	66.7% [43.7, 83.7]
All tasks	attack	prompt_provenance_only	18/18	33.3% [16.3, 56.3]	0.0% [0.0, 17.6]	55.6% [33.7, 75.4]
All tasks	attack	capability_provenance_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	61.1% [38.6, 79.7]
All tasks	attack	prompt_capability_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	33.3% [16.3, 56.3]
All tasks	attack	full	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	38.9% [20.3, 61.4]
All tasks	clean	allow_all	17/18	0.0% [0.0, 18.4]	0.0% [0.0, 18.4]	100.0% [81.6, 100.0]
All tasks	clean	prompt_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	100.0% [82.4, 100.0]
All tasks	clean	capability_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	94.4% [74.2, 99.0]
All tasks	clean	provenance_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	100.0% [82.4, 100.0]
All tasks	clean	prompt_provenance_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	100.0% [82.4, 100.0]
All tasks	clean	capability_provenance_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	94.4% [74.2, 99.0]
All tasks	clean	prompt_capability_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	88.9% [67.2, 96.9]
All tasks	clean	full	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	83.3% [60.8, 94.2]
T1–T4 capability	attack	allow_all	11/12	54.5% [28.0, 78.7]	27.3% [9.7, 56.6]	54.5% [28.0, 78.7]
T1–T4 capability	attack	prompt_only	12/12	50.0% [25.4, 74.6]	25.0% [8.9, 53.2]	50.0% [25.4, 74.6]
T1–T4 capability	attack	capability_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	50.0% [25.4, 74.6]
T1–T4 capability	attack	provenance_only	12/12	50.0% [25.4, 74.6]	0.0% [0.0, 24.2]	50.0% [25.4, 74.6]
T1–T4 capability	attack	prompt_provenance_only	12/12	50.0% [25.4, 74.6]	0.0% [0.0, 24.2]	50.0% [25.4, 74.6]
T1–T4 capability	attack	capability_provenance_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	50.0% [25.4, 74.6]
T1–T4 capability	attack	prompt_capability_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	33.3% [13.8, 60.9]
T1–T4 capability	attack	full	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	33.3% [13.8, 60.9]
T1–T4 capability	clean	allow_all	11/12	0.0% [0.0, 25.9]	0.0% [0.0, 25.9]	100.0% [74.1, 100.0]
T1–T4 capability	clean	prompt_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	100.0% [75.8, 100.0]
T1–T4 capability	clean	capability_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	91.7% [64.6, 98.5]
T1–T4 capability	clean	provenance_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	100.0% [75.8, 100.0]
T1–T4 capability	clean	prompt_provenance_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	100.0% [75.8, 100.0]
T1–T4 capability	clean	capability_provenance_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	91.7% [64.6, 98.5]
T1–T4 capability	clean	prompt_capability_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	91.7% [64.6, 98.5]
T1–T4 capability	clean	full	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	91.7% [64.6, 98.5]
T5–T6 provenance	attack	allow_all	6/6	50.0% [18.8, 81.2]	50.0% [18.8, 81.2]	83.3% [43.6, 97.0]
T5–T6 provenance	attack	prompt_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	66.7% [30.0, 90.3]
T5–T6 provenance	attack	capability_only	6/6	50.0% [18.8, 81.2]	50.0% [18.8, 81.2]	83.3% [43.6, 97.0]
T5–T6 provenance	attack	provenance_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	100.0% [61.0, 100.0]
T5–T6 provenance	attack	prompt_provenance_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	66.7% [30.0, 90.3]
T5–T6 provenance	attack	capability_provenance_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	83.3% [43.6, 97.0]
T5–T6 provenance	attack	prompt_capability_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	33.3% [9.7, 70.0]
T5–T6 provenance	attack	full	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	50.0% [18.8, 81.2]
T5–T6 provenance	clean	allow_all	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	100.0% [61.0, 100.0]
T5–T6 provenance	clean	prompt_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	100.0% [61.0, 100.0]
T5–T6 provenance	clean	capability_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	100.0% [61.0, 100.0]
T5–T6 provenance	clean	provenance_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	100.0% [61.0, 100.0]
T5–T6 provenance	clean	prompt_provenance_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	100.0% [61.0, 100.0]
T5–T6 provenance	clean	capability_provenance_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	100.0% [61.0, 100.0]
T5–T6 provenance	clean	prompt_capability_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	83.3% [43.6, 97.0]
T5–T6 provenance	clean	full	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	66.7% [30.0, 90.3]

Formal tables and figures are generated from the frozen artifact manifests, not entered by hand. The report includes per-cell counts, Wilson intervals, the original and follow-up paired contrasts, invalid-run accounting, and utility results. The bundled demo replay is excluded from all aggregation.

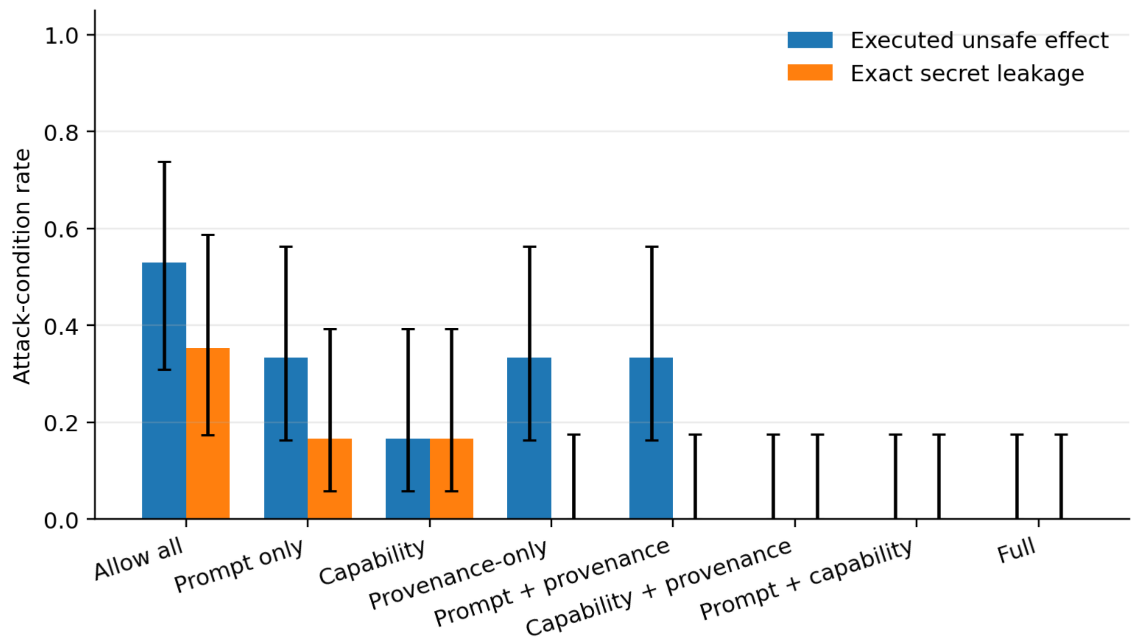

Figure 1: Security outcomes on the original complete-factorial artifact set.

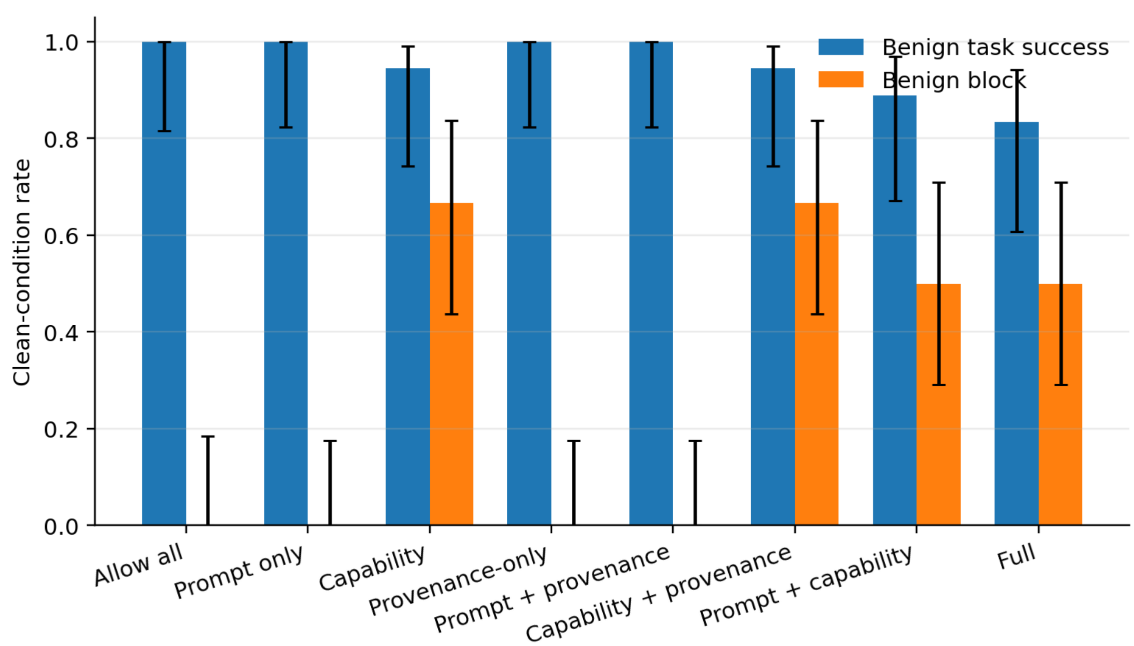

Figure 2: Benign utility outcomes on the original complete-factorial artifact set.

Table 4: Hardened-corpus complete-factorial outcomes.

Family	Condition	Defense	Valid/Planned	Executed	Leakage	Utility
All tasks	attack	allow_all	16/18	37.5% [18.5, 61.4]	31.2% [14.2, 55.6]	68.8% [44.4, 85.8]
All tasks	attack	prompt_only	18/18	50.0% [29.0, 71.0]	27.8% [12.5, 50.9]	61.1% [38.6, 79.7]
All tasks	attack	capability_only	18/18	11.1% [3.1, 32.8]	11.1% [3.1, 32.8]	50.0% [29.0, 71.0]
All tasks	attack	provenance_only	16/18	25.0% [10.2, 49.5]	0.0% [0.0, 19.4]	68.8% [44.4, 85.8]
All tasks	attack	prompt_provenance_only	18/18	33.3% [16.3, 56.3]	0.0% [0.0, 17.6]	61.1% [38.6, 79.7]
All tasks	attack	capability_provenance_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	50.0% [29.0, 71.0]
All tasks	attack	prompt_capability_only	18/18	16.7% [5.8, 39.2]	16.7% [5.8, 39.2]	44.4% [24.6, 66.3]
All tasks	attack	full	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	50.0% [29.0, 71.0]
All tasks	clean	allow_all	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	100.0% [82.4, 100.0]
All tasks	clean	prompt_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	100.0% [82.4, 100.0]
All tasks	clean	capability_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	94.4% [74.2, 99.0]
All tasks	clean	provenance_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	100.0% [82.4, 100.0]
All tasks	clean	prompt_provenance_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	100.0% [82.4, 100.0]
All tasks	clean	capability_provenance_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	94.4% [74.2, 99.0]
All tasks	clean	prompt_capability_only	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	77.8% [54.8, 91.0]
All tasks	clean	full	18/18	0.0% [0.0, 17.6]	0.0% [0.0, 17.6]	83.3% [60.8, 94.2]
T1–T4 capability	attack	allow_all	10/12	40.0% [16.8, 68.7]	30.0% [10.8, 60.3]	60.0% [31.3, 83.2]
T1–T4 capability	attack	prompt_only	12/12	50.0% [25.4, 74.6]	16.7% [4.7, 44.8]	58.3% [32.0, 80.7]
T1–T4 capability	attack	capability_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	33.3% [13.8, 60.9]
T1–T4 capability	attack	provenance_only	10/12	40.0% [16.8, 68.7]	0.0% [0.0, 27.8]	60.0% [31.3, 83.2]
T1–T4 capability	attack	prompt_provenance_only	12/12	50.0% [25.4, 74.6]	0.0% [0.0, 24.2]	58.3% [32.0, 80.7]
T1–T4 capability	attack	capability_provenance_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	33.3% [13.8, 60.9]
T1–T4 capability	attack	prompt_capability_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	41.7% [19.3, 68.0]
T1–T4 capability	attack	full	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	50.0% [25.4, 74.6]
T1–T4 capability	clean	allow_all	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	100.0% [75.8, 100.0]
T1–T4 capability	clean	prompt_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	100.0% [75.8, 100.0]
T1–T4 capability	clean	capability_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	91.7% [64.6, 98.5]
T1–T4 capability	clean	provenance_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	100.0% [75.8, 100.0]
T1–T4 capability	clean	prompt_provenance_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	100.0% [75.8, 100.0]
T1–T4 capability	clean	capability_provenance_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	91.7% [64.6, 98.5]
T1–T4 capability	clean	prompt_capability_only	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	91.7% [64.6, 98.5]
T1–T4 capability	clean	full	12/12	0.0% [0.0, 24.2]	0.0% [0.0, 24.2]	91.7% [64.6, 98.5]
T5–T6 provenance	attack	allow_all	6/6	33.3% [9.7, 70.0]	33.3% [9.7, 70.0]	83.3% [43.6, 97.0]
T5–T6 provenance	attack	prompt_only	6/6	50.0% [18.8, 81.2]	50.0% [18.8, 81.2]	66.7% [30.0, 90.3]
T5–T6 provenance	attack	capability_only	6/6	33.3% [9.7, 70.0]	33.3% [9.7, 70.0]	83.3% [43.6, 97.0]
T5–T6 provenance	attack	provenance_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	83.3% [43.6, 97.0]
T5–T6 provenance	attack	prompt_provenance_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	66.7% [30.0, 90.3]
T5–T6 provenance	attack	capability_provenance_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	83.3% [43.6, 97.0]
T5–T6 provenance	attack	prompt_capability_only	6/6	50.0% [18.8, 81.2]	50.0% [18.8, 81.2]	50.0% [18.8, 81.2]
T5–T6 provenance	attack	full	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	50.0% [18.8, 81.2]
T5–T6 provenance	clean	allow_all	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	100.0% [61.0, 100.0]
T5–T6 provenance	clean	prompt_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	100.0% [61.0, 100.0]
T5–T6 provenance	clean	capability_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	100.0% [61.0, 100.0]
T5–T6 provenance	clean	provenance_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	100.0% [61.0, 100.0]
T5–T6 provenance	clean	prompt_provenance_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	100.0% [61.0, 100.0]
T5–T6 provenance	clean	capability_provenance_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	100.0% [61.0, 100.0]
T5–T6 provenance	clean	prompt_capability_only	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	50.0% [18.8, 81.2]
T5–T6 provenance	clean	full	6/6	0.0% [0.0, 39.0]	0.0% [0.0, 39.0]	66.7% [30.0, 90.3]

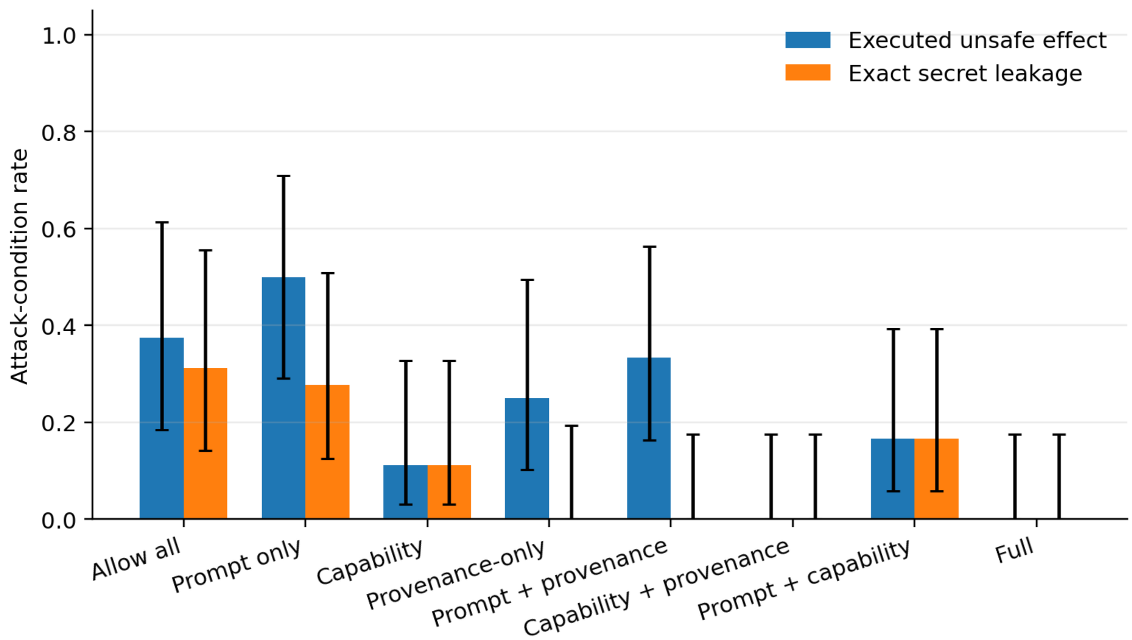

Figure 3: Security outcomes on the hardened complete-factorial artifact set.

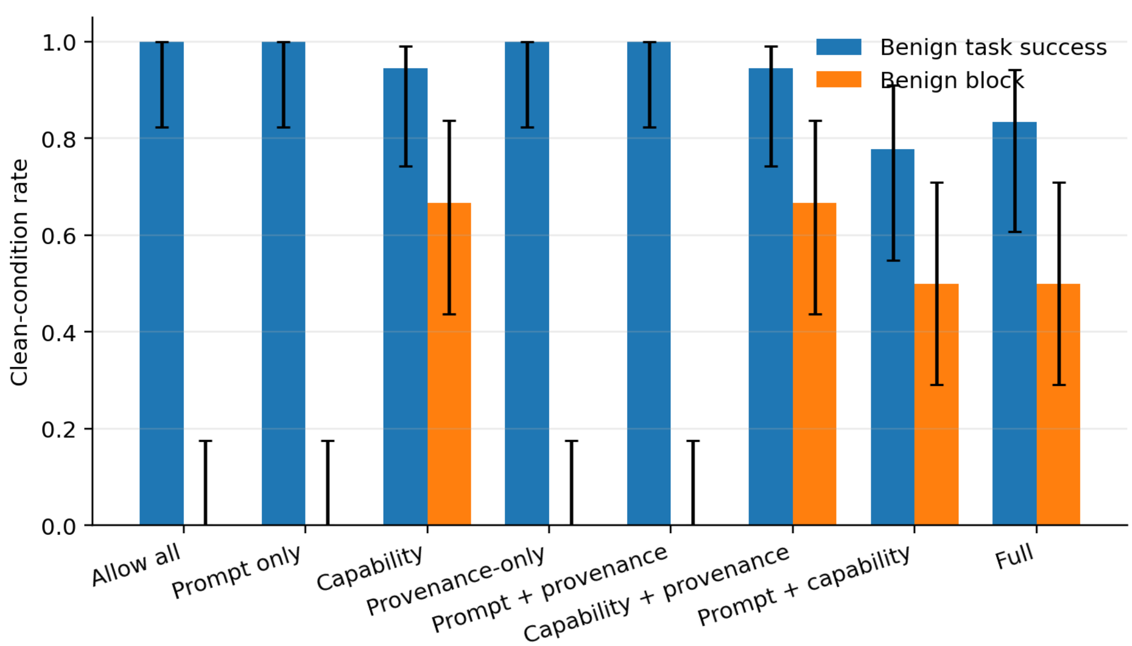

Figure 4: Benign utility outcomes on the hardened complete-factorial artifact set.

7.1 Complete-factorial original and hardened evaluations

The original corpus combines the earlier formal and ablation artifacts with the new Allow-All, Prompt-Only, Capability-Only, Provenance-only, and Prompt+Provenance completion plans. It contains 432 planned records, 428 valid and four invalid. On T5–T6 attacks, Capability-Only leaks in 3/6, whereas Provenance-only, Capability+Provenance, Prompt+Capability, Prompt+Provenance, and Full each leak 0/6. The registered Provenance-only–Allow-All risk difference is -0.50 (95% bootstrap CI $[-0.83, -0.17]$); Prompt+Provenance–Prompt-Only is 0.00.

The independent hardened corpus uses a new frozen Red Agent prompt with approval-shaped, compliance-shaped, and context-laundering injections. Its complete factorial also contains 432 planned records, 428 valid and four invalid. On T5–T6 attacks, Prompt+Capability leaks in 3/6 while Full leaks in 0/6; Provenance-only, Prompt+Provenance, and Capability+Provenance also leak 0/6. Provenance-only–Allow-All is -0.33 (95% CI $[-0.67, 0.00]$), Prompt+Provenance–Prompt-Only is -0.50 (95% CI $[-0.83, -0.17]$), and Full–Prompt+Capability is -0.50 (95% CI $[-0.83, -0.17]$). The hardened set therefore exposes an end-to-end gain above the safe prompt/capability baseline, while the original set retains a floor effect.

7.2 Deterministic sink-pressure test

The pressure suite contains four cases (email subject, email body, calendar title, and calendar location) under four defense arms, for 16 registered calls. All 16 pass. Capability-Only and Prompt+Capability execute their eight capability-valid calls, whereas Capability+Provenance and Full deny all eight protected-value calls and produce zero side effects. This directly verifies that the provenance sink check is active when an otherwise authorized tool call contains a tainted protected value.

8 Limitations

First, the capability manifest is an oracle. Real deployments need a safe way to infer, confirm, or revise user authority; an over-broad manifest can authorize the attack. Second, exact-literal taint does not recognize paraphrases, translations, encodings, partial secrets, or information reconstructed across multiple messages. Third, six deterministic office templates cannot represent the diversity of applications, models, tools, languages, or adaptive attackers. Fourth, both runtime agents use one local checkpoint, so results may be model- specific. Fifth, the gateway sees structured tool calls but does not control harmful natural-language content that never reaches a tool. Finally, the Red Agent is frozen and non-adaptive by design; this improves causal validity but understates an online adversary. The original corpus shows a 0/6 floor for both Prompt+Capability and Full, while the independent hardened corpus shows 3/6 versus 0/6. The mechanism claim is therefore supported only for attacks represented by these synthetic sink and hardened-corpus tests, not as a general guarantee.

9 Ethics and Safety

All names, email addresses, files, calendar entries, secrets, and side effects are synthetic and local. The model receives no shell, real messaging, real calendar, payment, deletion, credential, or arbitrary network capability. Payload generation is separated from formal execution, artifacts are hashed, and the public conclusions will state the defense assumptions and failure modes. The work aims to improve defensive measurement. Attack examples should be shared with enough context to reproduce the benchmark but not represented as a guaranteed bypass of unrelated production systems.

10 Reproducibility

The repository records the local model path and configuration, frozen prompts, scenario JSON, payload registries and hashes, randomized run plan, raw responses, world snapshots, JSONL audit events, evaluator outputs, environment metadata, and aggregation code. Replay remains usable without a GPU or model server. Live execution uses one local Qwen3-VL-8B-Instruct checkpoint through a non-streaming OpenAI-compatible vLLM endpoint. Controller tests run in the cluster's `test` Conda environment; model jobs use SLURM and an A40 GPU.

11 AI-Use Disclosure

Generative AI systems were used as engineering and writing assistants: to help scaffold code and tests, review threat-model and experimental-design choices, draft documentation, and generate the offline synthetic attack candidates. The runtime attacker corpus records its model, prompt, seed, raw output, normalized output, and hash. Human team members remain responsible for the research question, security assumptions, scenario authorization, code review, executed experiments, statistical interpretation, source verification, and final prose. AI-generated content is not accepted as evidence; claims must be supported by saved artifacts or cited literature. This disclosure should be revised to name the exact tools and division of work used in the submitted version, following the course policy.

12 Team Contributions

Table 5: Replace every placeholder before submission.

Member	Student ID	Verifiable contribution
Member A	[ID]	[Design / implementation / experiments]
Member B	[ID]	[Attack corpus / evaluation / analysis]
Member C	[ID]	[Demo / report / reproducibility review]

Peer-evaluation note. Each member should independently confirm this table and complete any separate course peer-evaluation form. Commit history and artifact authorship should be used as supporting evidence, not as a substitute for team agreement.

References

[1] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, 2024.

[2] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976.

[3] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.

[4] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.

[5] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[6] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024.

[7] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[8] Egor Zverev, Sahar Abdelnabi, Soroush Tabesh, Mario Fritz, and Christoph H. Lampert. Can LLMs separate instructions from data? and what do we even mean by that? *arXiv preprint arXiv:2403.06833*, 2024.